

2.多线程-初阶

本节目标

- 认识多线程
- 掌握多线程程序的编写
- 掌握多线程的状态
- 掌握什么是线程不安全及解决思路
- 掌握 synchronized、volatile 关键字

1. 认识线程 (Thread)

1.1 概念

1) 线程是什么

一个线程就是一个 "执行流". 每个线程之间都可以按照顺序执行自己的代码. 多个线程之间 "同时" 执行着多份代码.

还是回到我们之前的银行的例子中。之前我们主要描述的是个人业务，即一个人完全处理自己的业务。我们进一步设想如下场景：

一家公司要去银行办理业务，既要进行财务转账，又要进行福利发放，还得进行缴社保。

如果只有张三一个会计就会忙不过来，耗费的时间特别长。为了让业务更快的办理好，张三又找来两位同事李四、王五一起来帮助他，三个人分别负责一个事情，分别申请一个号码进行排队，自此就有了三个执行流共同完成任务，但本质上他们都是为了办理一家公司的业务。

此时，我们就把这种情况称为多线程，将一个大任务分解成不同小任务，交给不同执行流就分别排队执行。其中李四、王五都是张三叫来的，所以张三一般被称为主线程（Main Thread）。

2) 为啥要有线程

首先, "并发编程" 成为 "刚需".

- 单核 CPU 的发展遇到了瓶颈. 要想提高算力, 就需要多核 CPU. 而并发编程能更充分利用多核 CPU 资源.
- 有些任务场景需要 "等待 IO", 为了让等待 IO 的时间能够去做一些其他的工作, 也需要用到并发编程.

其次, 虽然多进程也能实现 并发编程, 但是线程比进程更轻量.

- 创建线程比创建进程更快.
- 销毁线程比销毁进程更快.
- 调度线程比调度进程更快.

最后, 线程虽然比进程轻量, 但是人们还不满足, 于是就有了 "线程池"(ThreadPool) 和 "协程"(Coroutine)

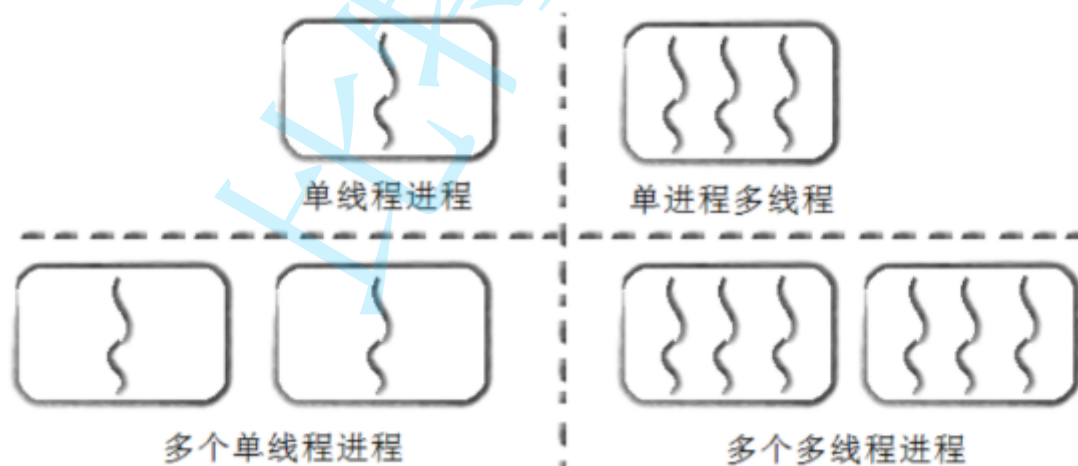
关于线程池我们后面再介绍. 关于协程的话题我们此处暂时不做过多讨论.

3) 进程和线程的区别

- 进程是包含线程的. 每个进程至少有一个线程存在, 即主线程。
- 进程和进程之间不共享内存空间. 同一个进程的线程之间共享同一个内存空间.

比如之前的多进程例子中, 每个客户来银行办理各自的业务, 但他们之间的票据肯定是不想让别人知道的, 否则钱不就被其他人取走了么。而上面我们的公司业务中, 张三、李四、王五虽然是不同的执行流, 但因为办理的都是一家公司的业务, 所以票据是共享着的。这个就是多线程和多进程的最大区别。

- 进程是系统分配资源的最小单位, 线程是系统调度的最小单位。
- 一个进程挂了一般不会影响到其他进程. 但是一个线程挂了, 可能把同进程内的其他线程一起带走(整个进程崩溃).



4) Java 的线程 和 操作系统线程 的关系

线程是操作系统中的概念. 操作系统内核实现了线程这样的机制, 并且对用户层提供了一些 API 供用户使用(例如 Linux 的 pthread 库).

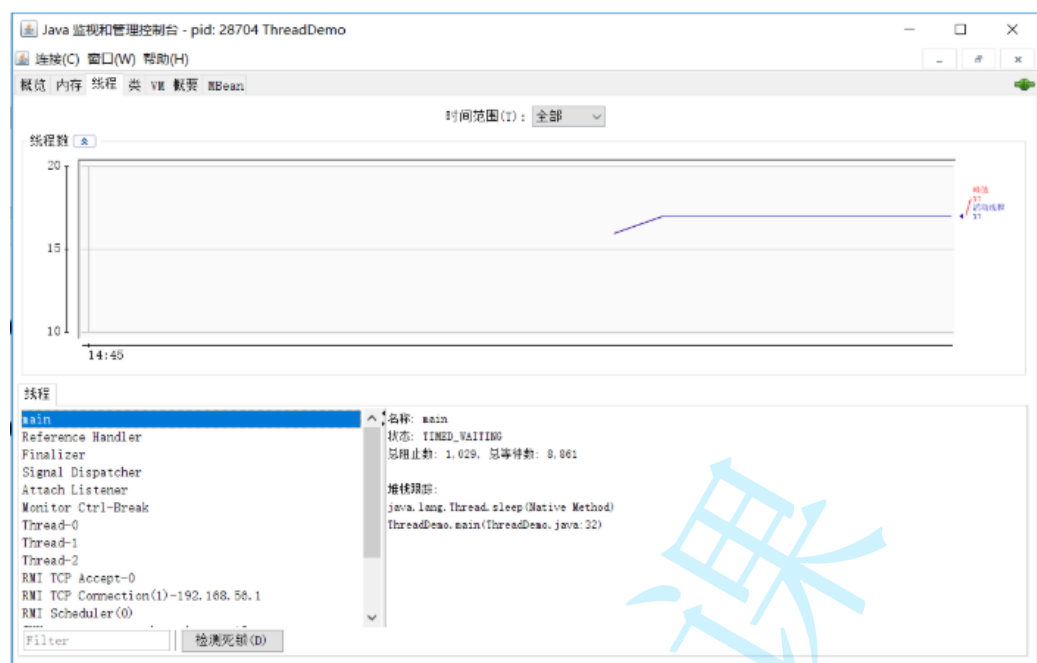
Java 标准库中 Thread 类可以视为是对操作系统提供的 API 进行了进一步的抽象和封装.

1.2 第一个多线程程序

感受多线程程序和普通程序的区别:

- 每个线程都是一个独立的执行流
- 多个线程之间是 "并发" 执行的.

```
1 import java.util.Random;
2
3 public class ThreadDemo {
4     private static class MyThread extends Thread {
5         @Override
6         public void run() {
7             Random random = new Random();
8             while (true) {
9                 // 打印线程名称
10                System.out.println(Thread.currentThread().getName());
11                try {
12                    // 随机停止运行 0-9 秒
13                    Thread.sleep(random.nextInt(10));
14                } catch (InterruptedException e) {
15                    e.printStackTrace();
16                }
17            }
18        }
19    }
20
21    public static void main(String[] args) {
22        MyThread t1 = new MyThread();
23        t1.start();
24
25        Random random = new Random();
26        while (true) {
27            // 打印线程名称
28            System.out.println(Thread.currentThread().getName());
29            try {
30                Thread.sleep(random.nextInt(10));
31            } catch (InterruptedException e) {
32                // 随机停止运行 0-9 秒
33                e.printStackTrace();
34            }
35        }
36    }
37 }
```



1.3 创建线程

方法1 继承 Thread 类

继承 Thread 来创建一个线程类.

```
1 class MyThread extends Thread {  
2     @Override  
3     public void run() {  
4         System.out.println("这里是线程运行的代码");  
5     }  
6 }
```

创建 MyThread 类的实例

```
1 MyThread t = new MyThread();
```

调用 start 方法启动线程

```
1 t.start();           // 线程开始运行
```

方法2 实现 Runnable 接口

1. 实现 Runnable 接口

```
1 class MyRunnable implements Runnable {
2     @Override
3     public void run() {
4         System.out.println("这里是线程运行的代码");
5     }
6 }
```

2. 创建 Thread 类实例, 调用 Thread 的构造方法时将 Runnable 对象作为 target 参数.

```
1 Thread t = new Thread(new MyRunnable());
```

3. 调用 start 方法

```
1 t.start();           // 线程开始运行
```

对比上面两种方法:

- 继承 Thread 类, 直接使用 this 就表示当前线程对象的引用.
- 实现 Runnable 接口, this 表示的是 MyRunnable 的引用. 需要使用 `Thread.currentThread()`

其他变形

- 匿名内部类创建 Thread 子类对象

```
1 // 使用匿名类创建 Thread 子类对象
2 Thread t1 = new Thread() {
3     @Override
4     public void run() {
5         System.out.println("使用匿名类创建 Thread 子类对象");
6     }
7 };
```

- 匿名内部类创建 Runnable 子类对象

```
1 // 使用匿名类创建 Runnable 子类对象
2 Thread t2 = new Thread(new Runnable() {
3     @Override
4     public void run() {
5         System.out.println("使用匿名类创建 Runnable 子类对象");
6     }
7 });
```

- lambda 表达式创建 Runnable 子类对象

```
1 // 使用 lambda 表达式创建 Runnable 子类对象
2 Thread t3 = new Thread(() -> System.out.println("使用匿名类创建 Thread 子类对象"));
3 Thread t4 = new Thread(() -> {
4     System.out.println("使用匿名类创建 Thread 子类对象");
5 });
```

1.4 多线程的优势-增加运行速度

可以观察多线程在一些场合下是可以提高程序的整体运行效率的。

- 使用 `System.nanoTime()` 可以记录当前系统的纳秒级时间戳。
- `serial` 串行的完成一系列运算. `concurrency` 使用两个线程并行的完成同样的运算。

```
1 public class ThreadAdvantage {
2     // 多线程并不一定就能提高速度，可以观察，count 不同，实际的运行效果也是不同的
3     private static final long count = 10_0000_0000;
4
5     public static void main(String[] args) throws InterruptedException {
6         // 使用并发方式
7         concurrency();
8         // 使用串行方式
9         serial();
10    }
11
12    private static void concurrency() throws InterruptedException {
13        long begin = System.nanoTime();
14
15        // 利用一个线程计算 a 的值
16        Thread thread = new Thread(new Runnable() {
17            @Override
```

```

18         public void run() {
19             int a = 0;
20             for (long i = 0; i < count; i++) {
21                 a--;
22             }
23         }
24     });
25     thread.start();
26     // 主线程内计算 b 的值
27     int b = 0;
28     for (long i = 0; i < count; i++) {
29         b--;
30     }
31     // 等待 thread 线程运行结束
32     thread.join();
33
34     // 统计耗时
35     long end = System.nanoTime();
36     double ms = (end - begin) * 1.0 / 1000 / 1000;
37     System.out.printf("并发: %f 毫秒\n", ms);
38 }
39
40 private static void serial() {
41     // 全部在主线程内计算 a、b 的值
42     long begin = System.nanoTime();
43     int a = 0;
44     for (long i = 0; i < count; i++) {
45         a--;
46     }
47     int b = 0;
48     for (long i = 0; i < count; i++) {
49         b--;
50     }
51     long end = System.nanoTime();
52     double ms = (end - begin) * 1.0 / 1000 / 1000;
53     System.out.printf("串行: %f 毫秒\n", ms);
54 }
55 }

```

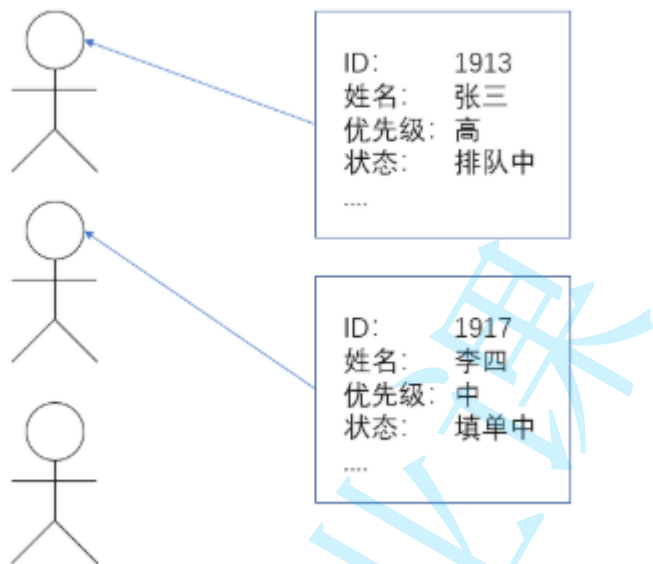
并发: 399.651856 毫秒

串行: 720.616911 毫秒

2. Thread 类及常见方法

Thread 类是 JVM 用来管理线程的一个类，换句话说，每个线程都有一个唯一的 Thread 对象与之关联。

用我们上面的例子来看，每个执行流，也需要有一个对象来描述，类似下图所示，而 Thread 类的对象就是用来描述一个线程执行流的，JVM 会将这些 Thread 对象组织起来，用于线程调度，线程管理。



2.1 Thread 的常见构造方法

方法	说明
Thread()	创建线程对象
Thread(Runnable target)	使用 Runnable 对象创建线程对象
Thread(String name)	创建线程对象，并命名
Thread(Runnable target, String name)	使用 Runnable 对象创建线程对象，并命名
【了解】 Thread(ThreadGroup group, Runnable target)	线程可以被用来分组管理，分好的组即为这个目前我们了解即可

```
1 Thread t1 = new Thread();
2 Thread t2 = new Thread(new MyRunnable());
3 Thread t3 = new Thread("这是我的名字");
4 Thread t4 = new Thread(new MyRunnable(), "这是我的名字");
```

2.2 Thread 的几个常见属性

属性	获取方法
ID	getId()
名称	getName()
状态	getState()
优先级	getPriority()
是否后台线程	isDaemon()
是否存活	isAlive()
是否被中断	isInterrupted()

- ID 是线程的唯一标识，不同线程不会重复
- 名称是各种调试工具用到
- 状态表示线程当前所处的一个情况，下面我们会进一步说明
- 优先级高的线程理论上来说更容易被调度到
- 关于后台线程，需要记住一点：**JVM会在一个进程的所有非后台线程结束后，才会结束运行。**
- 是否存活，即简单的理解，为 run 方法是否运行结束了
- 线程的中断问题，下面我们进一步说明

```
1 public class ThreadDemo {
2     public static void main(String[] args) {
3         Thread thread = new Thread(() -> {
4             for (int i = 0; i < 10; i++) {
5                 try {
6                     System.out.println(Thread.currentThread().getName() + ": 我过");
7                     Thread.sleep(1 * 1000);
8                 } catch (InterruptedException e) {
9                     e.printStackTrace();
10                }
11            }
12            System.out.println(Thread.currentThread().getName() + ": 我即将死去");
13        });
14
15        System.out.println(Thread.currentThread().getName()
16                            + ": ID: " + thread.getId());
17        System.out.println(Thread.currentThread().getName()
18                            + ": 名称: " + thread.getName());
19        System.out.println(Thread.currentThread().getName()
20                            + ": 状态: " + thread.getState());
21        System.out.println(Thread.currentThread().getName()
```

```

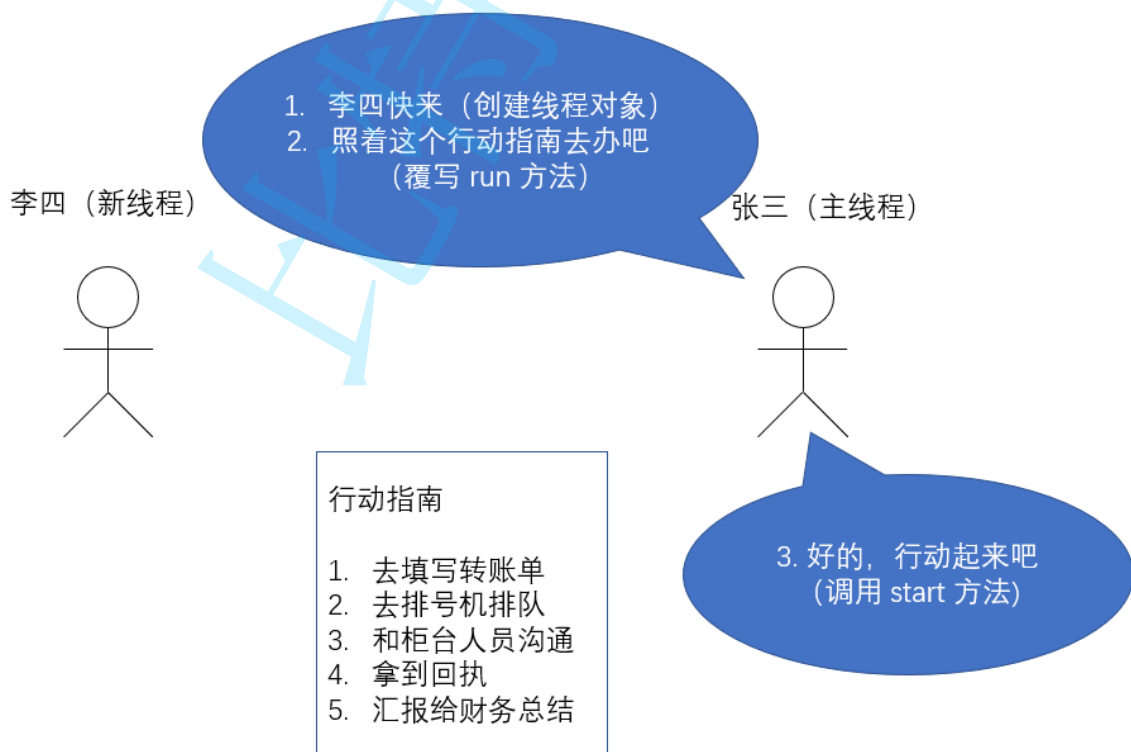
22         + ": 优先级: " + thread.getPriority());
23     System.out.println(Thread.currentThread().getName()
24         + ": 后台线程: " + thread.isDaemon());
25     System.out.println(Thread.currentThread().getName()
26         + ": 活着: " + thread.isAlive());
27     System.out.println(Thread.currentThread().getName()
28         + ": 被中断: " + thread.isInterrupted());
29
30     thread.start();
31     while (thread.isAlive()) {}
32     System.out.println(Thread.currentThread().getName()
33         + ": 状态: " + thread.getState());
34 }
35 }

```

2.3 启动一个线程 - start()

之前我们已经看到了如何通过覆写 run 方法创建一个线程对象，但线程对象被创建出来并不意味着线程就开始运行了。

- 覆写 run 方法是提供给线程要做的事情的指令清单
- 线程对象可以认为是把 李四、王五叫过来了
- 而调用 start() 方法，就是喊一声：“行动起来！”，线程才真正独立去执行了。



调用 start 方法，才真的在操作系统的底层创建一个线程。

2.4 中断一个线程

李四一旦进到工作状态，他就会按照行动指南上的步骤去进行工作，不完成是不会结束的。但有时我们需要增加一些机制，例如老板突然来电话了，说转账的对方是个骗子，需要赶紧停止转账，那张三该如何通知李四停止呢？这就涉及到我们的停止线程的方式了。

目前常见的有以下两种方式：

1. 通过共享的标记来进行沟通
2. 调用 `interrupt()` 方法来通知

示例-1: 使用自定义的变量来作为标志位。

需要给标志位上加 `volatile` 关键字(这个关键字的功能后面介绍)。

```
1 public class ThreadDemo {
2     private static class MyRunnable implements Runnable {
3         public volatile boolean isQuit = false;
4
5         @Override
6         public void run() {
7             while (!isQuit) {
8                 System.out.println(Thread.currentThread().getName()
9                     + ": 别管我，我忙着转账呢!");
10                try {
11                    Thread.sleep(1000);
12                } catch (InterruptedException e) {
13                    e.printStackTrace();
14                }
15            }
16            System.out.println(Thread.currentThread().getName()
17                + ": 啊！险些误了大事");
18        }
19    }
20
21    public static void main(String[] args) throws InterruptedException {
22        MyRunnable target = new MyRunnable();
23        Thread thread = new Thread(target, "李四");
24        System.out.println(Thread.currentThread().getName()
25            + ": 让李四开始转账。");
26        thread.start();
27        Thread.sleep(10 * 1000);
28    }
29 }
```

```

28         System.out.println(Thread.currentThread().getName()
29             + ": 老板来电话了, 得赶紧通知李四对方是个骗子!");
30         target.isQuit = true;
31     }
32 }

```

示例-2: 使用 `Thread.interrupted()` 或者

`Thread.currentThread().isInterrupted()` 代替自定义标志位。

`Thread` 内部包含了一个 `boolean` 类型的变量作为线程是否被中断的标记。

方法	说明
<code>public void interrupt()</code>	中断对象关联的线程，如果线程正在阻塞，则
<code>public static boolean interrupted()</code>	判断当前线程的中断标志位是否设置，调用后
<code>public boolean isInterrupted()</code>	判断对象关联的线程的标志位是否设置，调用后

- 使用 `Thread` 对象的 `interrupted()` 方法通知线程结束。

```

1 public class ThreadDemo {
2     private static class MyRunnable implements Runnable {
3         @Override
4         public void run() {
5             // 两种方法均可以
6             while (!Thread.interrupted()) {
7                 //while (!Thread.currentThread().isInterrupted()) {
8                 System.out.println(Thread.currentThread().getName()
9                     + ": 别管我, 我忙着转账呢!");
10                try {
11                    Thread.sleep(1000);
12                } catch (InterruptedException e) {
13                    e.printStackTrace();
14                    System.out.println(Thread.currentThread().getName()
15                        + ": 有内鬼, 终止交易!");
16                    // 注意此处的 break
17                    break;
18                }
19            }
20            System.out.println(Thread.currentThread().getName()
21                + ": 啊! 险些误了大事");
22        }
23    }
24 }

```

```

24
25     public static void main(String[] args) throws InterruptedException {
26         MyRunnable target = new MyRunnable();
27         Thread thread = new Thread(target, "李四");
28         System.out.println(Thread.currentThread().getName()
29             + ": 让李四开始转账。");
30         thread.start();
31         Thread.sleep(10 * 1000);
32         System.out.println(Thread.currentThread().getName()
33             + ": 老板来电话了, 得赶紧通知李四对方是个骗子!");
34         thread.interrupt();
35     }
36 }

```

thread 收到通知的方式有两种:

1. 如果线程因为调用 wait/join/sleep 等方法而阻塞挂起, 则以 InterruptedException 异常的形式通知, **清除中断标志**
 - 当出现 InterruptedException 的时候, 要不要结束线程取决于 catch 中代码的写法. 可以选择忽略这个异常, 也可以跳出循环结束线程.
2. 否则, 只是内部的一个中断标志被设置, thread 可以通过
 - Thread.currentThread().isInterrupted() 判断指定线程的中断标志被设置, **不清除中断标志**

这种方式通知收到的更及时, 即使线程正在 sleep 也可以马上收到。

2.5 等待一个线程 - join()

有时, 我们需要等待一个线程完成它的工作后, 才能进行自己的下一步工作。例如, 张三只有等李四转账成功, 才决定是否存钱, 这时我们需要一个方法明确等待线程的结束。

```

1 public class ThreadDemo {
2     public static void main(String[] args) throws InterruptedException {
3         Runnable target = () -> {
4             for (int i = 0; i < 10; i++) {
5                 try {
6                     System.out.println(Thread.currentThread().getName()
7                         + ": 我还在工作!");
8                     Thread.sleep(1000);
9                 } catch (InterruptedException e) {
10                    e.printStackTrace();
11                }
12            }
13            System.out.println(Thread.currentThread().getName() + ": 我结束了!");

```

```

14     };
15
16     Thread thread1 = new Thread(target, "李四");
17     Thread thread2 = new Thread(target, "王五");
18     System.out.println("先让李四开始工作");
19     thread1.start();
20     thread1.join();
21     System.out.println("李四工作结束了，让王五开始工作");
22     thread2.start();
23     thread2.join();
24     System.out.println("王五工作结束了");
25 }
26 }

```

大家可以试试如果把两个 join 注释掉，现象会是怎么样的呢？

附录

方法	说明
public void join()	等待线程结束
public void join(long millis)	等待线程结束，最多等 millis 毫秒
public void join(long millis, int nanos)	同理，但可以更高精度

2.6 获取当前线程引用

这个方法我们已经非常熟悉了

方法	说明
public static Thread currentThread();	返回当前线程对象的引用

```

1 public class ThreadDemo {
2     public static void main(String[] args) {
3         Thread thread = Thread.currentThread();
4         System.out.println(thread.getName());
5     }
6 }

```

2.7 休眠当前线程

也是我们比较熟悉一组方法，有一点要记得，因为线程的调度是不可控的，所以，这个方法只能保证实际休眠时间是大于等于参数设置的休眠时间的。

方法	说明
public static void sleep(long millis) throws InterruptedException	休眠当前线程 r
public static void sleep(long millis, int nanos) throws InterruptedException	可以更高精度的

```
1 public class ThreadDemo {
2     public static void main(String[] args) throws InterruptedException {
3         System.out.println(System.currentTimeMillis());
4         Thread.sleep(3 * 1000);
5         System.out.println(System.currentTimeMillis());
6     }
7 }
```

3. 线程的状态

3.1 观察线程的所有状态

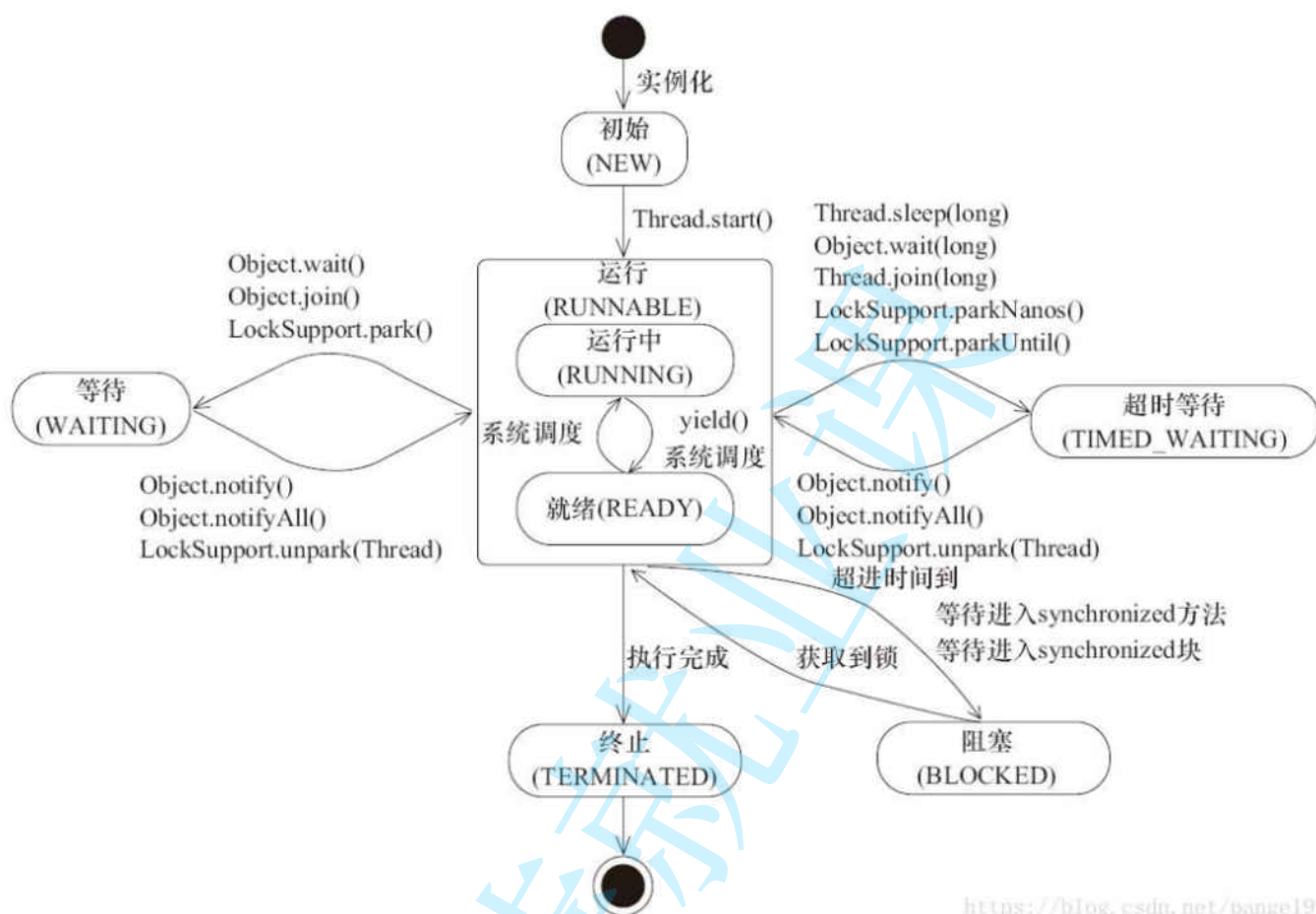
线程的状态是一个枚举类型 Thread.State

```
1 public class ThreadState {
2     public static void main(String[] args) {
3         for (Thread.State state : Thread.State.values()) {
4             System.out.println(state);
5         }
6     }
7 }
```

- NEW: 安排了工作, 还未开始行动
- RUNNABLE: 可工作的. 又可以分成正在工作中和即将开始工作.
- BLOCKED: 这几个都表示排队等着其他事情
- WAITING: 这几个都表示排队等着其他事情
- TIMED_WAITING: 这几个都表示排队等着其他事情

- TERMINATED: 工作完成了.

3.2 线程状态和状态转移的意义



<https://blog.csdn.net/pange1991>

大家不要被这个状态转移图吓到，我们重点是要理解状态的意义以及各个状态的具体意思。



还是我们之前的例子：

刚把李四、王五找来，还是给他们在安排任务，没让他们行动起来，就是 NEW 状态；

当李四、王五开始去窗口排队，等待服务，就进入到 **RUNNABLE** 状态。该状态并不表示已经被银行工作人员开始接待，排在队伍中也是属于该状态，即可被服务的状态，是否开始服务，则看调度器的调度；

当李四、王五因为一些事情需要去忙，例如需要填写信息、回家取证件、发呆一会等等时，进入 **BLOCKED**、**WAITING**、**TIMED_WAITING** 状态，至于这些状态的细分，我们以后再详解；

如果李四、王五已经忙完，为 **TERMINATED** 状态。

所以，之前我们学过的 **isAlive()** 方法，可以认为是处于不是 **NEW** 和 **TERMINATED** 的状态都是活着的。

3.3 观察线程的状态和转移

观察 1: 关注 **NEW**、**RUNNABLE**、**TERMINATED** 状态的转换

```
1 public class ThreadStateTransfer {
2     public static void main(String[] args) throws InterruptedException {
3         Thread t = new Thread(() -> {
4             for (int i = 0; i < 1000_0000; i++) {
```

```

5         }
6     }, "李四");
7
8     System.out.println(t.getName() + ": " + t.getState());
9     t.start();
10    while (t.isAlive()) {
11        System.out.println(t.getName() + ": " + t.getState());
12    }
13    System.out.println(t.getName() + ": " + t.getState());
14 }
15 }

```

观察 2: 关注 WAITING 、 BLOCKED 、 TIMED_WAITING 状态的转换

```

1 public static void main(String[] args) {
2     final Object object = new Object();
3
4     Thread t1 = new Thread(new Runnable() {
5         @Override
6         public void run() {
7             synchronized (object) {
8                 while (true) {
9                     try {
10                        Thread.sleep(1000);
11                    } catch (InterruptedException e) {
12                        e.printStackTrace();
13                    }
14                }
15            }
16        }
17    }, "t1");
18    t1.start();
19
20    Thread t2 = new Thread(new Runnable() {
21        @Override
22        public void run() {
23            synchronized (object) {
24                System.out.println("hehe");
25            }
26        }
27    }, "t2");
28    t2.start();
29 }

```

使用 jconsole 可以看到 t1 的状态是 TIMED_WAITING , t2 的状态是 BLOCKED

修改上面的代码, 把 t1 中的 sleep 换成 wait

```
1 public static void main(String[] args) {
2     final Object object = new Object();
3
4     Thread t1 = new Thread(new Runnable() {
5         @Override
6         public void run() {
7             synchronized (object) {
8                 try {
9                     // [修改这里就可以了!!!!]
10                    // Thread.sleep(1000);
11                    object.wait();
12                } catch (InterruptedException e) {
13                    e.printStackTrace();
14                }
15            }
16        }
17    }, "t1");
18    ...
19 }
```

使用 jconsole 可以看到 t1 的状态是 WAITING

结论:

- BLOCKED 表示等待获取锁, WAITING 和 TIMED_WAITING 表示等待其他线程发来通知.
- TIMED_WAITING 线程在等待唤醒, 但设置了时限; WAITING 线程在无限等待唤醒

4. 多线程带来的风险-线程安全 (重点)

4.1 观察线程不安全

```
1 // 此处定义一个 int 类型的变量
2 private static int count = 0;
3
4 public static void main(String[] args) throws InterruptedException {
5     Thread t1 = new Thread(() -> {
```

```

6      // 对 count 变量进行自增 5w 次
7      for (int i = 0; i < 50000; i++) {
8          count++;
9      }
10     });
11     Thread t2 = new Thread(() -> {
12         // 对 count 变量进行自增 5w 次
13         for (int i = 0; i < 50000; i++) {
14             count++;
15         }
16     });
17
18     t1.start();
19     t2.start();
20
21     // 如果没有这俩 join, 肯定不行的。线程还没自增完, 就开始打印了。很可能打印出来的 cou
22     t1.join();
23     t2.join();
24
25     // 预期结果应该是 10w
26     System.out.println("count: " + count);
27 }

```

大家观察下是否适用多线程的现象是否一致？同时尝试思考下为什么会有这样的现象发生呢？

4.2 线程安全的概念

想给出一个线程安全的确切定义是复杂的，但我们可以这样认为：

如果多线程环境下代码运行的结果是符合我们预期的，即在单线程环境应该的结果，则说这个程序是线程安全的。

4.3 线程不安全的原因

线程调度是随机的

这是线程安全问题的罪魁祸首

随机调度使一个程序在多线程环境下, 执行顺序存在很多的变数.

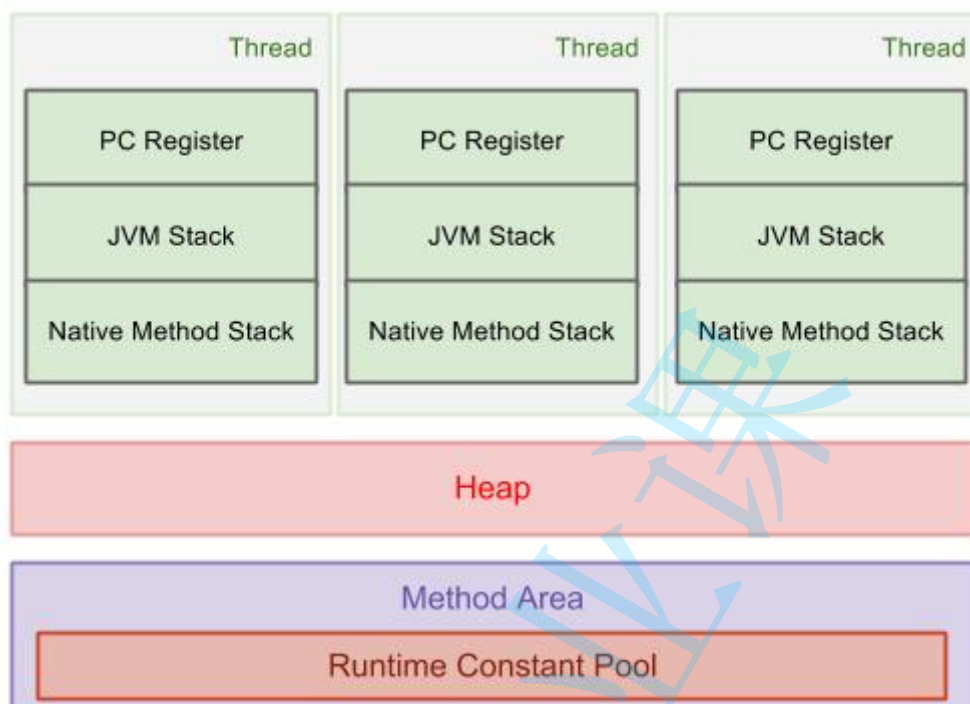
程序猿必须保证 在任意执行顺序下, 代码都能正常工作.

修改共享数据

多个线程修改同一个变量

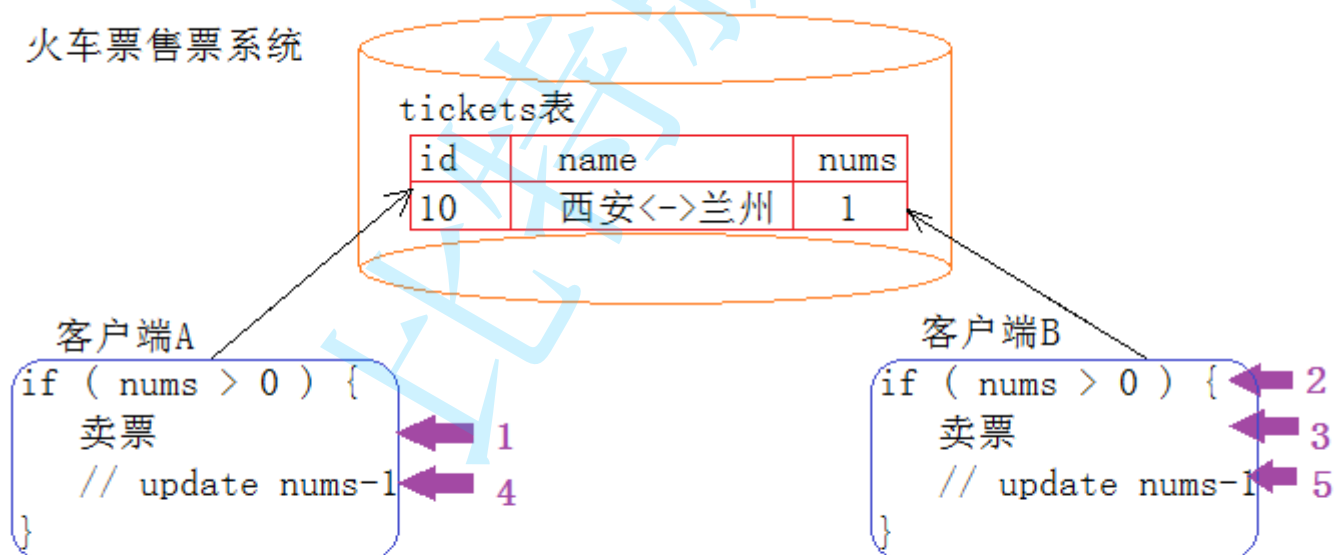
上面的线程不安全的代码中, 涉及到多个线程针对 `count` 变量进行修改。

此时这个 `count` 是一个多个线程都能访问到的 "共享数据"



原子性

火车票售票系统



当客户端A检查还有一张票时, 将票卖掉, 还没有执行更新数据库时, 客户端B检查了票数, 发现大于0, 于是又卖了一次票。然后A将票数更新回数据库。于是就出现了同一张票被卖了两次。

什么是原子性

我们把一段代码想象成一个房间, 每个线程就是要进入这个房间的人。如果没有任何机制保证, A进入房间之后, 还没有出来; B是不是也可以进入房间, 打断A在房间里的隐私。这个就是不具备原子性

的。

那我们应该如何解决这个问题呢？是不是只要给房间加一把锁，A 进去就把门锁上，其他人是不是就进不来了。这样就保证了这段代码的原子性了。

有时也把这个现象叫做同步互斥，表示操作是互相排斥的。

一条 java 语句不一定是原子的，也不一定只是一条指令

比如刚才我们看到的 `n++`，其实是由三步操作组成的：

1. 从内存把数据读到 CPU
2. 进行数据更新
3. 把数据写回到 CPU

不保证原子性会给多线程带来什么问题

如果一个线程正在对一个变量操作，中途其他线程插入进来了，如果这个操作被打断了，结果就可能是错误的。

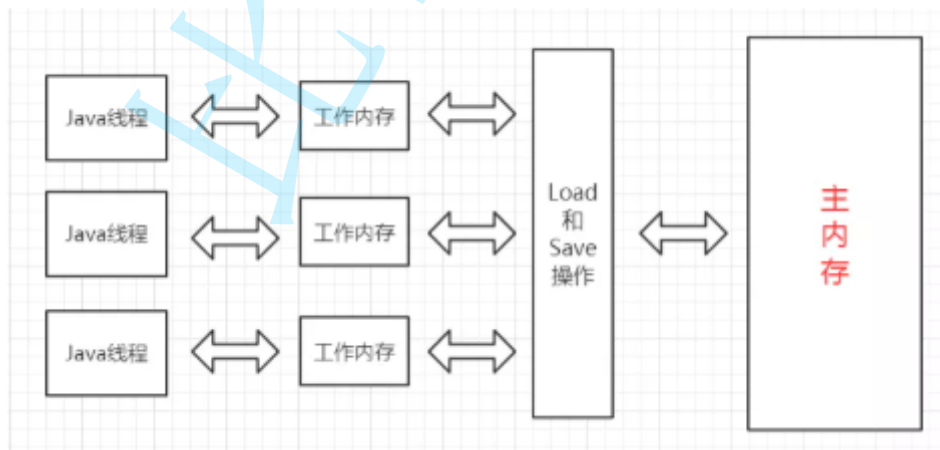
这点也和线程的抢占式调度密切相关. 如果线程不是 "抢占" 的, 就算没有原子性, 也问题不大.

可见性

可见性指, 一个线程对共享变量值的修改，能够及时地被其他线程看到。

Java 内存模型 (JMM): Java虚拟机规范中定义了Java内存模型。

目的是屏蔽掉各种硬件和操作系统的内存访问差异，以实现让Java程序在各种平台下都能达到一致的并发效果。

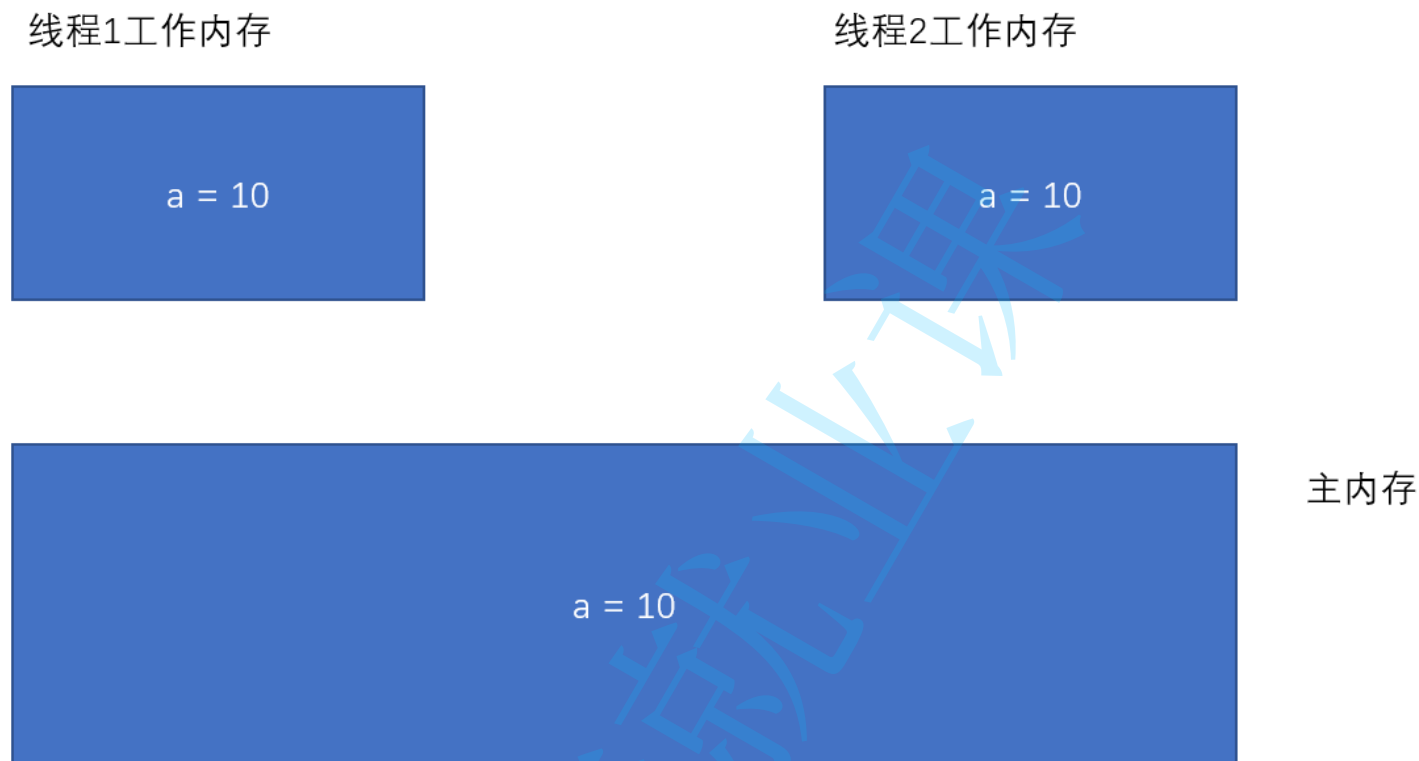


- 线程之间的共享变量存在 主内存 (Main Memory).
- 每一个线程都有自己的 "工作内存" (Working Memory) .
- 当线程要读取一个共享变量的时候, 会先把变量从主内存拷贝到工作内存, 再从工作内存读取数据.

- 当线程要修改一个共享变量的时候, 也会先修改工作内存中的副本, 再同步回主内存.

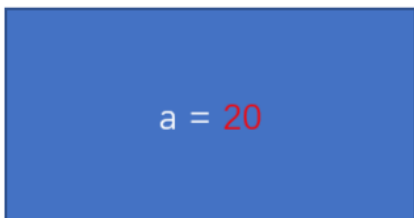
由于每个线程有自己的工作内存, 这些工作内存中的内容相当于同一个共享变量的 "副本". 此时修改线程1 的工作内存中的值, 线程2 的工作内存不一定会及时变化.

1) 初始情况下, 两个线程的工作内存内容一致.

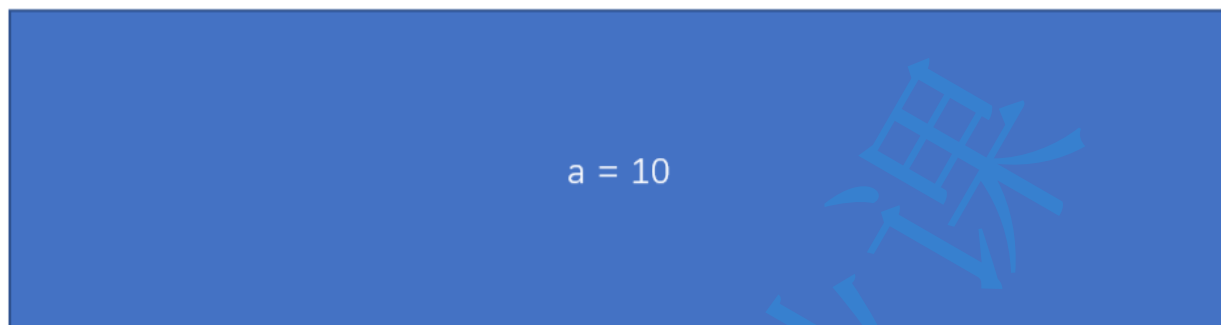
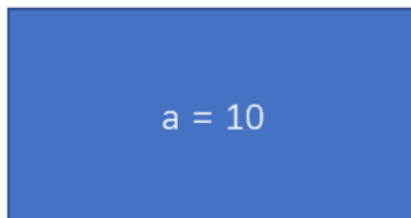


2) 一旦线程1 修改了 a 的值, 此时主内存不一定能及时同步. 对应的线程2 的工作内存的 a 的值也不一定能及时同步.

线程1工作内存



线程2工作内存



主内存

这个时候代码中就容易出现问题.

此时引入了两个问题:

- 为啥要整这么多内存?
- 为啥要这么麻烦的拷来拷去?

1) 为啥整这么多内存?

实际并没有这么多 "内存". 这只是 Java 规范中的一个术语, 是属于 "抽象" 的叫法.

所谓的 "主内存" 才是真正硬件角度的 "内存". 而所谓的 "工作内存", 则是指 CPU 的寄存器和高速缓存.

2) 为啥要这么麻烦的拷来拷去?

因为 CPU 访问自身寄存器的速度以及高速缓存的速度, 远远超过访问内存的速度(快了 3 - 4 个数量级, 也就是几千倍, 上万倍).

比如某个代码中要连续 10 次读取某个变量的值, 如果 10 次都从内存读, 速度是很慢的. 但是如果只是第一次从内存读, 读到的结果缓存到 CPU 的某个寄存器中, 那么后 9 次读数据就不必直接访问内存了. 效率就大大提高了.

那么接下来问题又来了, 既然访问寄存器速度这么快, 还要内存干啥??

答案就是一个字: 贵



来自贫穷的凝视

值得一提的是, 快和慢都是相对的. CPU 访问寄存器速度远远快于内存, 但是内存的访问速度又远远快于硬盘.

对应的, CPU 的价格最贵, 内存次之, 硬盘最便宜.

指令重排序

什么是代码重排序

一段代码是这样的:

1. 去前台取下 U 盘
2. 去教室写 10 分钟作业
3. 去前台取下快递

如果是在单线程情况下, JVM、CPU 指令集会对其进行优化, 比如, 按 1->3->2 的方式执行, 也是没问题, 可以少跑一次前台. 这种叫做指令重排序

编译器对于指令重排序的前提是 "保持逻辑不发生变化". 这一点在单线程环境下比较容易判断, 但是在多线程环境下就没那么容易了, 多线程的代码执行复杂程度更高, 编译器很难在编译阶段对代码的执行效果进行预测, 因此激进的重排序很容易导致优化后的逻辑和之前不等价.

重排序是一个比较复杂的话题, 涉及到 CPU 以及编译器的一些底层工作原理, 此处不做过多讨论

4.4 解决之前的线程不安全问题

这里用到的机制, 我们马上会给大家解释。

```
1 // 此处定义一个 int 类型的变量
2 private static int count = 0;
```

```

3
4 public static void main(String[] args) throws InterruptedException {
5     Object locker = new Object();
6
7     Thread t1 = new Thread(() -> {
8         // 对 count 变量进行自增 5w 次
9         for (int i = 0; i < 50000; i++) {
10             synchronized (locker) {
11                 count++;
12             }
13         }
14     });
15     Thread t2 = new Thread(() -> {
16         // 对 count 变量进行自增 5w 次
17         for (int i = 0; i < 50000; i++) {
18             synchronized (locker) {
19                 count++;
20             }
21         }
22     });
23
24     t1.start();
25     t2.start();
26
27     // 如果没有这俩 join, 肯定不行的. 线程还没自增完, 就开始打印了. 很可能打印出来的 cou
28     t1.join();
29     t2.join();
30
31     // 预期结果应该是 10w
32     System.out.println("count: " + count);
33 }

```

5. synchronized 关键字 - 监视器锁 monitor lock

5.1 synchronized 的特性

1) 互斥

synchronized 会起到互斥效果, 某个线程执行到某个对象的 synchronized 中时, 其他线程如果也执行到同一个对象 synchronized 就会**阻塞等待**.

- 进入 synchronized 修饰的代码块, 相当于 **加锁**
- 退出 synchronized 修饰的代码块, 相当于 **解锁**

进入方法内部, 相当于
针对当前对象 "加锁"

```
synchronized void increase() {  
    count++;  
}
```

执行方法完毕相当于
针对当前对象 "解锁"

synchronized用的锁是存在Java对象头里的。

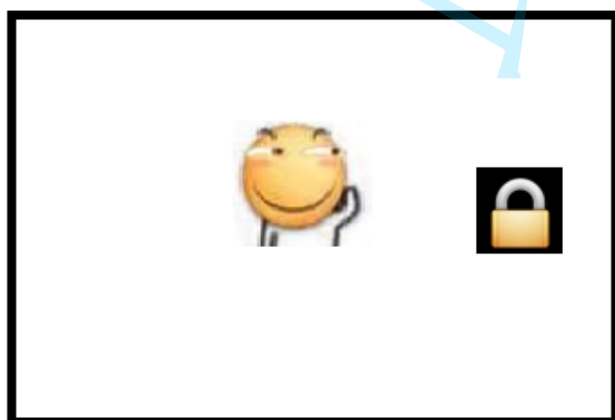


可以粗略理解成, 每个对象在内存中存储的时候, 都存有一块内存表示当前的 "锁定" 状态(类似于厕所的 "有人/无人").

如果当前是 "无人" 状态, 那么就可以使用, 使用时需要设为 "有人" 状态.

如果当前是 "有人" 状态, 那么其他人无法使用, 只能排队

对象



排队等待

一个线程先上了锁, 其他线程只能等待这个线程释放

理解 "阻塞等待".

针对每一把锁, 操作系统内部都维护了一个等待队列. 当这个锁被某个线程占有的时候, 其他线程尝试进行加锁, 就加不上了, 就会阻塞等待, 一直等到之前的线程解锁之后, 由操作系统唤醒一个新的线程, 再来获取到这个锁.

注意:

- 上一个线程解锁之后, 下一个线程并不是立即就能获取到锁. 而是要靠操作系统来 "唤醒". 这也就是操作系统线程调度的一部分工作.
- 假设有 A B C 三个线程, 线程 A 先获取到锁, 然后 B 尝试获取锁, 然后 C 再尝试获取锁, 此时 B 和 C 都在阻塞队列中排队等待. 但是当 A 释放锁之后, 虽然 B 比 C 先来的, 但是 B 不一定就能获取到锁, 而是和 C 重新竞争, 并不遵守先来后到的规则.

synchronized的底层是使用操作系统的mutex lock实现的.

2) 可重入

synchronized 同步块对同一条线程来说是可重入的, 不会出现自己把自己锁死的问题;

理解 "把自己锁死"

一个线程没有释放锁, 然后又尝试再次加锁.

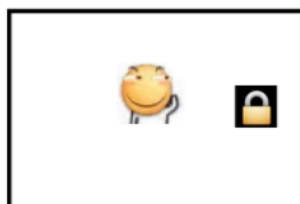
// 第一次加锁, 加锁成功

lock();

// 第二次加锁, 锁已经被占用, 阻塞等待.

lock();

按照之前对于锁的设定, 第二次加锁的时候, 就会阻塞等待. 直到第一次的锁被释放, 才能获取到第二个锁. 但是释放第一个锁也是由该线程来完成, 结果这个线程已经躺平了, 啥都不想干了, 也就无法进行解锁操作. 这时候就会 **死锁**.



进了 WC, 要把门锁上



时空扭曲
滑稽释放了 "闪现"
并获得了 "失忆" buff



滑稽: 里面这是哪个 SB 这么久了还不出来?

这样的锁称为 **不可重入锁**.

Java 中的 synchronized 是 **可重入锁**, 因此没有上面的问题.

```

1 for (int i = 0; i < 50000; i++) {
2     synchronized (locker) {
3         synchronized (locker) {
4             count++;
5         }
6     }
7 }

```

在可重入锁的内部, 包含了 "线程持有者" 和 "计数器" 两个信息.

- 如果某个线程加锁的时候, 发现锁已经被别人占用, 但是恰好占用的正是自己, 那么仍然可以继续获取到锁, 并让计数器自增.
- 解锁的时候计数器递减为 0 的时候, 才真正释放锁. (才能被别的线程获取到)

5.2 synchronized 使用示例

synchronized 本质上要修改指定对象的 "对象头". 从使用角度来看, synchronized 也势必要搭配一个具体的对象来使用.

1) 修饰代码块: 明确指定锁哪个对象.

锁任意对象

```

1 public class SynchronizedDemo {
2     private Object locker = new Object();
3
4     public void method() {
5         synchronized (locker) {
6
7         }
8     }
9 }

```

锁当前对象

```

1 public class SynchronizedDemo {
2     public void method() {
3         synchronized (this) {
4

```

```
5     }  
6     }  
7 }
```

2) 直接修饰普通方法: 锁的 SynchronizedDemo 对象

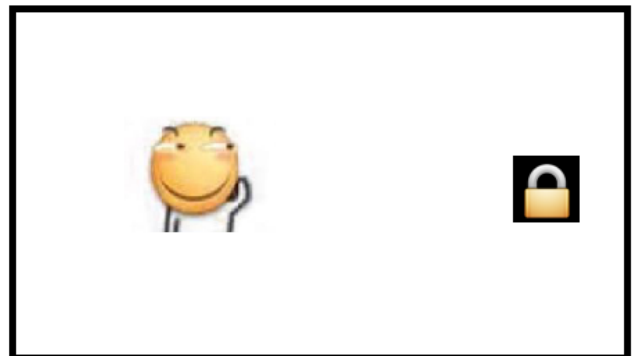
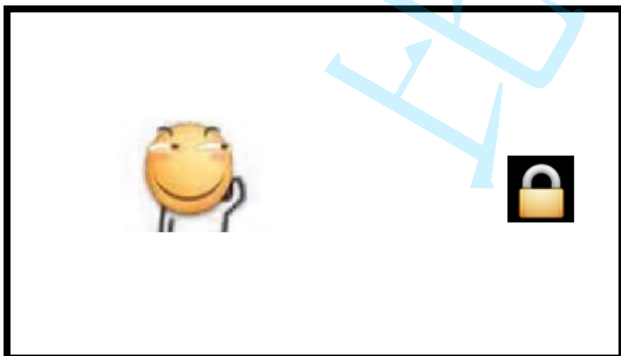
```
1 public class SynchronizedDemo {  
2     public synchronized void method() {  
3     }  
4 }
```

3) 修饰静态方法: 锁的 SynchronizedDemo 类的对象

```
1 public class SynchronizedDemo {  
2     public synchronized static void method() {  
3     }  
4 }
```

我们重点要理解, **synchronized** 锁的是**什么**. 两个线程竞争同一把锁, 才会产生阻塞等待.

两个线程分别尝试获取两把不同的锁, 不会产生竞争.



5.3 Java 标准库中的线程安全类

Java 标准库中很多都是线程不安全的. 这些类可能会涉及到多线程修改共享数据, 又没有任何加锁措施.

- ArrayList

- LinkedList
- HashMap
- TreeMap
- HashSet
- TreeSet
- StringBuilder

但是还有一些是线程安全的. 使用了一些锁机制来控制.

- Vector (不推荐使用)
- Hashtable (不推荐使用)
- ConcurrentHashMap
- StringBuffer

```
@Override
public synchronized StringBuffer append(Object obj) {
    toStringCache = null;
    super.append(String.valueOf(obj));
    return this;
}
```

StringBuffer 的核心方法都带有 `synchronized` .

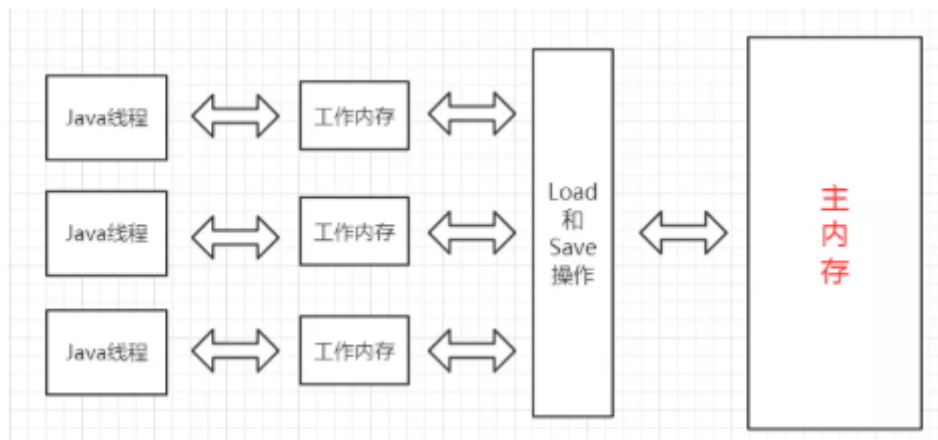
还有的虽然没有加锁, 但是不涉及 "修改", 仍然是线程安全的

- String

6. volatile 关键字

volatile 能保证内存可见性

volatile 修饰的变量, 能够保证 "内存可见性".



代码在写入 volatile 修饰的变量的时候,

- 改变线程工作内存中volatile变量副本的值
- 将改变后的副本的值从工作内存刷新到主内存

代码在读取 volatile 修饰的变量的时候,

- 从主内存中读取volatile变量的最新值到线程的工作内存中
- 从工作内存中读取volatile变量的副本

前面我们讨论内存可见性时说了, 直接访问工作内存(实际是 CPU 的寄存器或者 CPU 的缓存), 速度非常快, 但是可能出现数据不一致的情况.

加上 volatile , 强制读写内存. 速度是慢了, 但是数据变的更准确了.



代码示例

在这个代码中

- 创建两个线程 t1 和 t2
- t1 中包含一个循环, 这个循环以 `flag == 0` 为循环条件.
- t2 中从键盘读入一个整数, 并把这个整数赋值给 `flag`.

- 预期当用户输入非 0 的值的时候, t1 线程结束.

```
1 static class Counter {
2     public int flag = 0;
3 }
4
5 public static void main(String[] args) {
6     Counter counter = new Counter();
7     Thread t1 = new Thread(() -> {
8         while (counter.flag == 0) {
9             // do nothing
10        }
11        System.out.println("循环结束!");
12    });
13
14    Thread t2 = new Thread(() -> {
15        Scanner scanner = new Scanner(System.in);
16        System.out.println("输入一个整数:");
17        counter.flag = scanner.nextInt();
18    });
19
20    t1.start();
21    t2.start();
22 }
23
24 // 执行效果
25 // 当用户输入非0值时, t1 线程循环不会结束. (这显然是一个 bug)
```

t1 读的是自己工作内存中的内容.

当 t2 对 flag 变量进行修改, 此时 t1 感知不到 flag 的变化.

如果给 flag 加上 volatile

```
1 static class Counter {
2     public volatile int flag = 0;
3 }
4
5 // 执行效果
6 // 当用户输入非0值时, t1 线程循环能够立即结束.
```

volatile 不保证原子性

volatile 和 synchronized 有着本质的区别. synchronized 能够保证原子性, volatile 保证的是内存可见性.

代码示例

这个是最初的演示线程安全的代码.

- 给 increase 方法去掉 synchronized
- 给 count 加上 volatile 关键字.

```
1 static class Counter {
2     volatile public int count = 0;
3
4     void increase() {
5         count++;
6     }
7 }
8
9 public static void main(String[] args) throws InterruptedException {
10     final Counter counter = new Counter();
11
12     Thread t1 = new Thread(() -> {
13         for (int i = 0; i < 50000; i++) {
14             counter.increase();
15         }
16     });
17     Thread t2 = new Thread(() -> {
18         for (int i = 0; i < 50000; i++) {
19             counter.increase();
20         }
21     });
22     t1.start();
23     t2.start();
24     t1.join();
25     t2.join();
26
27     System.out.println(counter.count);
28 }
```

此时可以看到, 最终 count 的值仍然无法保证是 100000.

7. wait 和 notify

由于线程之间是抢占式执行的, 因此线程之间执行的先后顺序难以预知。

但是实际开发中有时候我们希望合理的协调多个线程之间的执行先后顺序。



球场上的每个运动员都是独立的 "执行流", 可以认为是一个 "线程".

而完成一个具体的进攻得分动作, 则需要多个运动员相互配合, 按照一定的顺序执行一定的动作, 线程 1 先 "传球", 线程 2 才能 "扣篮".

完成这个协调工作, 主要涉及到三个方法

- `wait()` / `wait(long timeout)`: 让当前线程进入等待状态.
- `notify()` / `notifyAll()`: 唤醒在当前对象上等待的线程.

注意: `wait`, `notify`, `notifyAll` 都是 `Object` 类的方法.

7.1 `wait()`方法

`wait` 做的事情:

- 使当前执行代码的线程进行等待. (把线程放到等待队列中)
- 释放当前的锁
- 满足一定条件时被唤醒, 重新尝试获取这个锁.

`wait` 要搭配 `synchronized` 来使用. 脱离 `synchronized` 使用 `wait` 会直接抛出异常.

`wait` 结束等待的条件:

- 其他线程调用该对象的 `notify` 方法.
- `wait` 等待时间超时 (`wait` 方法提供一个带有 `timeout` 参数的版本, 来指定等待时间).
- 其他线程调用该等待线程的 `interrupted` 方法, 导致 `wait` 抛出 `InterruptedException` 异常.

代码示例: 观察`wait()`方法使用

```
1 public static void main(String[] args) throws InterruptedException {
2     Object object = new Object();
3     synchronized (object) {
4         System.out.println("等待中");
5         object.wait();
6         System.out.println("等待结束");
7     }
8 }
```

这样在执行到`object.wait()`之后就一直等待下去，那么程序肯定不能一直这么等待下去了。这个时候就需要使用到了另外一个方法唤醒的方法`notify()`。

7.2 notify()方法

`notify` 方法是唤醒等待的线程。

- 方法`notify()`也要在同步方法或同步块中调用，该方法是用来通知那些可能等待该对象的对象锁的其它线程，对其发出通知`notify`，并使它们重新获取该对象的对象锁。
- 如果有多个线程等待，则有线程调度器随机挑选出一个呈 `wait` 状态的线程。(并没有 "先来后到")
- 在`notify()`方法后，当前线程不会马上释放该对象锁，要等到执行`notify()`方法的线程将程序执行完，也就是退出同步代码块之后才会释放对象锁。

代码示例: 使用`notify()`方法唤醒线程

- 创建 `WaitTask` 类, 对应一个线程, `run` 内部循环调用 `wait`.
- 创建 `NotifyTask` 类, 对应另一个线程, 在 `run` 内部调用一次 `notify`
- 注意, `WaitTask` 和 `NotifyTask` 内部持有同一个 `Object locker`. `WaitTask` 和 `NotifyTask` 要想配合就需要搭配同一个 `Object`.

```
1 static class WaitTask implements Runnable {
2     private Object locker;
3
4     public WaitTask(Object locker) {
5         this.locker = locker;
6     }
7
8     @Override
9     public void run() {
10         synchronized (locker) {
11             while (true) {
```

```

12         try {
13             System.out.println("wait 开始");
14             locker.wait();
15             System.out.println("wait 结束");
16         } catch (InterruptedException e) {
17             e.printStackTrace();
18         }
19     }
20 }
21 }
22 }
23
24 static class NotifyTask implements Runnable {
25     private Object locker;
26
27     public NotifyTask(Object locker) {
28         this.locker = locker;
29     }
30
31     @Override
32     public void run() {
33         synchronized (locker) {
34             System.out.println("notify 开始");
35             locker.notify();
36             System.out.println("notify 结束");
37         }
38     }
39 }
40
41 public static void main(String[] args) throws InterruptedException {
42     Object locker = new Object();
43     Thread t1 = new Thread(new WaitTask(locker));
44     Thread t2 = new Thread(new NotifyTask(locker));
45     t1.start();
46     Thread.sleep(1000);
47     t2.start();
48 }

```

7.3 notifyAll()方法

notify方法只是唤醒某一个等待线程. 使用notifyAll方法可以一次唤醒所有的等待线程.

范例：使用notifyAll()方法唤醒所有等待线程, 在上面的代码基础上做出修改.

- 创建 3 个 WaitTask 实例. 1 个 NotifyTask 实例.

```
1 static class WaitTask implements Runnable {
2     // 代码不变
3 }
4
5 static class NotifyTask implements Runnable {
6     // 代码不变
7 }
8
9 public static void main(String[] args) throws InterruptedException {
10     Object locker = new Object();
11     Thread t1 = new Thread(new WaitTask(locker));
12     Thread t3 = new Thread(new WaitTask(locker));
13     Thread t4 = new Thread(new WaitTask(locker));
14     Thread t2 = new Thread(new NotifyTask(locker));
15     t1.start();
16     t3.start();
17     t4.start();
18     Thread.sleep(1000);
19     t2.start();
20 }
```

此时可以看到, 调用 `notify` 只能唤醒一个线程.

- 修改 NotifyTask 中的 run 方法, 把 `notify` 替换成 `notifyAll`

```
1 public void run() {
2     synchronized (locker) {
3         System.out.println("notify 开始");
4         locker.notifyAll();
5         System.out.println("notify 结束");
6     }
7 }
```

此时可以看到, 调用 `notifyAll` 能同时唤醒 3 个wait 中的线程

注意: 虽然是同时唤醒 3 个线程, 但是这 3 个线程需要竞争锁. 所以并不是同时执行, 而仍然是有先有后的执行.

理解 `notify` 和 `notifyAll`

`notify` 只唤醒等待队列中的一个线程. 其他线程还是乖乖等着



面试房间

兄弟们, 俺先进去
给你们探探路哈



应聘者在排队

加油嗷老哥~



notifyAll 一下全都唤醒, 需要这些线程重新竞争锁



面试房间



放开, 让我先进去



GNMD, 让劳资先来



你们这群辣鸡都闪开



明明是我先来的 T-T

7.4 wait 和 sleep 的对比（面试题）

其实理论上 wait 和 sleep 完全是没有可比性的, 因为一个是用于线程之间的通信的, 一个是让线程阻塞一段时间,

唯一的相同点就是都可以让线程放弃执行一段时间.

当然为了面试的目的, 我们还是总结下:

1. wait 需要搭配 synchronized 使用. sleep 不需要.

2. wait 是 Object 的方法 sleep 是 Thread 的静态方法.

8. 多线程案例

8.1 单例模式

单例模式是校招中最常考的设计模式之一.

啥是设计模式?

设计模式好比象棋中的 "棋谱". 红方当头炮, 黑方马来跳. 针对红方的一些走法, 黑方应招的时候有一些固定的套路. 按照套路来走局势就不会吃亏.

软件开发中也有很多常见的 "问题场景". 针对这些问题场景, 大佬们总结出了一些固定的套路. 按照这个套路来实现代码, 也不会吃亏.

单例模式能保证某个类在程序中只存在唯一一份实例, 而不会创建出多个实例.

这一点在很多场景上都需要. 比如 JDBC 中的 DataSource 实例就只需要一个.

单例模式具体的实现方式有很多. 最常见的是 "饿汉" 和 "懒汉" 两种.

饿汉模式

类加载的同时, 创建实例.

```
1 class Singleton {
2     private static Singleton instance = new Singleton();
3     private Singleton() {}
4     public static Singleton getInstance() {
5         return instance;
6     }
7 }
```

懒汉模式-单线程版

类加载的时候不创建实例. 第一次使用的时候才创建实例.

```
1 class Singleton {
```



```
2     private static Singleton instance = null;
3     private Singleton() {}
4     public static Singleton getInstance() {
5         if (instance == null) {
6             instance = new Singleton();
7         }
8         return instance;
9     }
10 }
```

懒汉模式-多线程版

上面的懒汉模式的实现是线程不安全的。

线程安全问题发生在首次创建实例时. 如果在多个线程中同时调用 `getInstance` 方法, 就可能导致创建出多个实例.

一旦实例已经创建好了, 后面再多线程环境调用 `getInstance` 就不再有线程安全问题了(不再修改 `instance` 了)

加上 `synchronized` 可以改善这里的线程安全问题.

```
1 class Singleton {
2     private static Singleton instance = null;
3     private Singleton() {}
4     public synchronized static Singleton getInstance() {
5         if (instance == null) {
6             instance = new Singleton();
7         }
8         return instance;
9     }
10 }
```

懒汉模式-多线程版(改进)

以下代码在加锁的基础上, 做出了进一步改动:

- 使用双重 `if` 判定, 降低锁竞争的频率.
- 给 `instance` 加上了 `volatile`.

```
1 class Singleton {
```

```

2     private static volatile Singleton instance = null;
3     private Singleton() {}
4     public static Singleton getInstance() {
5         if (instance == null) {
6             synchronized (Singleton.class) {
7                 if (instance == null) {
8                     instance = new Singleton();
9                 }
10            }
11        }
12        return instance;
13    }
14 }

```

理解双重 if 判定 / volatile:

加锁 / 解锁是一件开销比较高的事情. 而懒汉模式的线程不安全只是发生在首次创建实例的时候. 因此后续使用的时候, 不必再进行加锁了.

外层的 if 就是判定下看当前是否已经把 instance 实例创建出来了.

同时为了避免 "内存可见性" 导致读取的 instance 出现偏差, 于是补充上 volatile .

当多线程首次调用 getInstance, 大家可能都发现 instance 为 null, 于是又继续往下执行来竞争锁, 其中竞争成功的线程, 再完成创建实例的操作.

当这个实例创建完了之后, 其他竞争到锁的线程就被里层 if 挡住了. 也就不会继续创建其他实例.

1. 有三个线程, 开始执行 `getInstance`, 通过外层的 `if (instance == null)` 知道了实例还没有创建的消息. 于是开始竞争同一把锁.



校花还没男票, 咱们
三兄弟各凭本事好嘛



追求校花的大军在路上



2. 其中线程1 率先获取到锁, 此时线程1 通过里层的 `if (instance == null)` 进一步确认实例是否已经创建. 如果没创建, 就把这个实例创建出来.



3. 当线程1 释放锁之后, 线程2 和 线程3 也拿到锁, 也通过里层的 `if (instance == null)` 来确认实例是否已经创建, 发现实例已经创建出来了, 就不再创建了.



4. 后续的线程, 不必加锁, 直接就通过外层 `if (instance == null)` 就知道实例已经创建了, 从而不再尝试获取锁了. 降低了开销.



听说校花已经有男票了?
那咱们就不进去问了吧, 散了散了~



8.2 阻塞队列

阻塞队列是什么

阻塞队列是一种特殊的队列. 也遵守 "先进先出" 的原则.

阻塞队列能是一种线程安全的数据结构, 并且具有以下特性:

- 当队列满的时候, 继续入队列就会阻塞, 直到有其他线程从队列中取走元素.
- 当队列空的时候, 继续出队列也会阻塞, 直到有其他线程往队列中插入元素.

阻塞队列的一个典型应用场景就是 "生产者消费者模型". 这是一种非常典型的开发模型.

生产者消费者模型

生产者消费者模式就是通过一个容器来解决生产者和消费者的强耦合问题.

生产者和消费者彼此之间不直接通讯, 而通过阻塞队列来进行通讯, 所以生产者生产完数据之后不用等待消费者处理, 直接扔给阻塞队列, 消费者不找生产者要数据, 而是直接从阻塞队列里取.

1. 阻塞队列就相当于一个缓冲区, 平衡了生产者和消费者的处理能力. (削峰填谷)

比如在 "秒杀" 场景下, 服务器同一时刻可能会收到大量的支付请求. 如果直接处理这些支付请求, 服务器可能扛不住(每个支付请求的处理都需要比较复杂的流程). 这个时候就可以把这些请求都放到一个阻塞队列中, 然后再由消费者线程慢慢的来处理每个支付请求.

这样做可以有效进行 "削峰", 防止服务器被突然到来的一波请求直接冲垮.

2. 阻塞队列也能使生产者和消费者之间 解耦.

比如过年一家人一起包饺子. 一般都是有明确分工, 比如一个人负责擀饺子皮, 其他人负责包. 擀饺子皮的人就是 "生产者", 包饺子的人就是 "消费者".

擀饺子皮的人不关心包饺子的人是谁(能包就行, 无论是手工包, 借助工具, 还是机器包), 包饺子的人也不关心擀饺子皮的人是谁(有饺子皮就行, 无论是用擀面杖擀的, 还是拿罐头瓶擀, 还是直接从超市买的).

标准库中的阻塞队列

在 Java 标准库中内置了阻塞队列. 如果我们需要在一些程序中使用阻塞队列, 直接使用标准库中的即可.

- BlockingQueue 是一个接口. 真正实现的类是 LinkedBlockingQueue.
- put 方法用于阻塞式的入队列, take 用于阻塞式的出队列.
- BlockingQueue 也有 offer, poll, peek 等方法, 但是这些方法不带有阻塞特性.

```
1 BlockingQueue<String> queue = new LinkedBlockingQueue<>();
2 // 入队列
3 queue.put("abc");
4 // 出队列. 如果没有 put 直接 take, 就会阻塞.
5 String elem = queue.take();
```

生产者消费者模型

```
1 public static void main(String[] args) throws InterruptedException {
2     BlockingQueue<Integer> blockingQueue = new LinkedBlockingQueue<Integer>();
3
4     Thread customer = new Thread(() -> {
5         while (true) {
6             try {
7                 int value = blockingQueue.take();
8                 System.out.println("消费元素: " + value);
9             } catch (InterruptedException e) {
10                 e.printStackTrace();
11             }
12         }
13     }, "消费者");
14     customer.start();
15
16     Thread producer = new Thread(() -> {
17         Random random = new Random();
18         while (true) {
```

```

19         try {
20             int num = random.nextInt(1000);
21             System.out.println("生产元素: " + num);
22             blockingQueue.put(num);
23             Thread.sleep(1000);
24         } catch (InterruptedException e) {
25             e.printStackTrace();
26         }
27     }
28 }, "生产者");
29 producer.start();
30
31 customer.join();
32 producer.join();
33 }

```

阻塞队列实现

- 通过 "循环队列" 的方式来实现.
- 使用 synchronized 进行加锁控制.
- put 插入元素的时候, 判定如果队列满了, 就进行 wait. (注意, 要在循环中进行 wait. 被唤醒时不一定队列就不满了, 因为同时可能是唤醒了多个线程).
- take 取出元素的时候, 判定如果队列为空, 就进行 wait. (也是循环 wait)

```

1 public class BlockingQueue {
2     private int[] items = new int[1000];
3     private volatile int size = 0;
4     private volatile int head = 0;
5     private volatile int tail = 0;
6
7     public void put(int value) throws InterruptedException {
8         synchronized (this) {
9             // 此处最好使用 while.
10            // 否则 notifyAll 的时候, 该线程从 wait 中被唤醒,
11            // 但是紧接着并未抢占到锁. 当锁被抢占的时候, 可能又已经队列满了
12            // 就只能继续等待
13            while (size == items.length) {
14                wait();
15            }
16            items[tail] = value;
17            tail = (tail + 1) % items.length;
18            size++;

```

```
19         notifyAll();
20     }
21 }
22
23 public int take() throws InterruptedException {
24     int ret = 0;
25     synchronized (this) {
26         while (size == 0) {
27             wait();
28         }
29         ret = items[head];
30         head = (head + 1) % items.length;
31         size--;
32         notifyAll();
33     }
34     return ret;
35 }
36
37 public synchronized int size() {
38     return size;
39 }
40
41 // 测试代码
42 public static void main(String[] args) throws InterruptedException {
43     BlockingQueue blockingQueue = new BlockingQueue();
44
45     Thread customer = new Thread(() -> {
46         while (true) {
47             try {
48                 int value = blockingQueue.take();
49                 System.out.println(value);
50             } catch (InterruptedException e) {
51                 e.printStackTrace();
52             }
53         }
54     }, "消费者");
55     customer.start();
56
57     Thread producer = new Thread(() -> {
58         Random random = new Random();
59         while (true) {
60             try {
61                 blockingQueue.put(random.nextInt(10000));
62             } catch (InterruptedException e) {
63                 e.printStackTrace();
64             }
65         }
66     });
```



```
66     }, "生产者");
67     producer.start();
68
69     customer.join();
70     producer.join();
71 }
72 }
```

8.3 定时器

定时器是什么

定时器也是软件开发中的一个重要组件. 类似于一个 "闹钟". 达到一个设定的时间之后, 就执行某个指定好的代码.



定时器是一种实际开发中非常常用的组件.

比如网络通信中, 如果对方 500ms 内没有返回数据, 则断开连接尝试重连.

比如一个 Map, 希望里面的某个 key 在 3s 之后过期(自动删除).

类似于这样的场景就需要用到定时器.

标准库中的定时器

- 标准库中提供了一个 Timer 类. Timer 类的核心方法为 `schedule`.
- `schedule` 包含两个参数. 第一个参数指定即将要执行的任务代码, 第二个参数指定多长时间之后执行 (单位为毫秒).


```

1 Timer timer = new Timer();
2 timer.schedule(new TimerTask() {
3     @Override
4     public void run() {
5         System.out.println("hello");
6     }
7 }, 3000);

```

实现定时器

定时器的构成

- 一个带优先级队列(不要使用 `PriorityBlockingQueue`, 容易死锁!)
- 队列中的每个元素是一个 Task 对象.
- Task 中带有时间属性, 队首元素就是即将要执行的任务
- 同时有一个 worker 线程一直扫描队首元素, 看队首元素是否需要执行

1. Timer 类提供的核心接口为 `schedule`, 用于注册一个任务, 并指定这个任务多长时间后执行.

```

1 public class MyTimer {
2     public void schedule(Runnable command, long after) {
3         // TODO
4     }
5 }

```

2. Task 类用于描述一个任务(作为 Timer 的内部类). 里面包含一个 Runnable 对象和一个 time(毫秒时间戳)

这个对象需要放到 优先队列 中. 因此需要实现 `Comparable` 接口.

```

1 class MyTask implements Comparable<MyTask> {
2     public Runnable runnable;
3     // 为了方便后续判定, 使用绝对的时间戳.
4     public long time;
5
6
7     public MyTask(Runnable runnable, long delay) {
8         this.runnable = runnable;
9     }
10
11     @Override
12     public int compareTo(MyTask other) {
13         return Long.compare(this.time, other.time);
14     }
15 }

```

```

9      // 取当前时刻的时间戳 + delay, 作为该任务实际执行的时间戳
10     this.time = System.currentTimeMillis() + delay;
11 }
12
13 @Override
14 public int compareTo(MyTask o) {
15     // 这样的写法意味着每次取出的是时间最小的元素。
16     // 到底是谁减谁?? 俺也记不住!!! 随便写一个, 执行下, 看看效果~~
17     return (int)(this.time - o.time);
18 }
19 }

```

3. Timer 实例中, 通过 PriorityQueue 来组织若干个 Task 对象.

通过 schedule 来往队列中插入一个个 Task 对象.

```

1 class MyTimer {
2     // 核心结构
3     private PriorityQueue<MyTask> queue = new PriorityQueue<>();
4     // 创建一个锁对象
5     private Object locker = new Object();
6
7     public void schedule(Runnable command, long after) {
8         // 根据参数, 构造 MyTask, 插入队列即可。
9         synchronized (locker) {
10             MyTask myTask = new MyTask(command, after);
11             queue.offer(myTask);
12             locker.notify();
13         }
14     }
15 }

```

4. Timer 类中存在一个 worker 线程, 一直不停的扫描队首元素, 看看是否能执行这个任务.

所谓 "能执行" 指的是该任务设定的时间已经到达了.

```

1 // 在这里构造线程, 负责执行具体任务了。
2 public MyTimer() {
3     Thread t = new Thread(() -> {
4         while (true) {
5             try {

```

```

6         synchronized (locker) {
7             // 阻塞队列，只有阻塞的入队列和阻塞的出队列，没有阻塞的查看队首元素
8             while (queue.isEmpty()) {
9                 locker.wait();
10            }
11            MyTask myTask = queue.peek();
12            long curTime = System.currentTimeMillis();
13            if (curTime >= myTask.time) {
14                // 时间到了，可以执行任务了
15                queue.poll();
16                myTask.runnable.run();
17            } else {
18                // 时间还没到
19                locker.wait(myTask.time - curTime);
20            }
21        }
22    } catch (InterruptedException e) {
23        e.printStackTrace();
24    }
25    }
26    });
27    t.start();
28 }

```

8.4 线程池

线程池是什么

虽然创建线程 / 销毁线程 的开销

想象这么一个场景：

在学校附近新开了一家快递店，老板很精明，想到一个与众不同的办法来经营。店里没有雇人，而是每次有业务来了，就现场找一名同学过来把快递送了，然后解雇同学。这个类比我们平时来一个任务，起一个线程进行处理的模式。

很快老板发现问题来了，每次招聘 + 解雇同学的成本还是非常高的。老板还是很善于变通的，知道了为什么大家都要雇人了，所以指定了一个指标，公司业务人员会扩张到 3 个人，但还是随着业务逐步雇人。于是再有业务来了，老板就看，如果现在公司还没 3 个人，就雇一个人去送快递，否则只是把业务放到一个本本上，等着 3 个快递人员空闲的时候去处理。这个就是我们要带出的线程池的模式。

线程池最大的好处就是减少每次启动、销毁线程的损耗。

标准库中的线程池

- 使用 `Executors.newFixedThreadPool(10)` 能创建出固定包含 10 个线程的线程池。
- 返回值类型为 `ExecutorService`
- 通过 `ExecutorService.submit` 可以注册一个任务到线程池中。

```
1 ExecutorService pool = Executors.newFixedThreadPool(10);
2 pool.submit(new Runnable() {
3     @Override
4     public void run() {
5         System.out.println("hello");
6     }
7 });
```

Executors 创建线程池的几种方式

- `newFixedThreadPool`: 创建固定线程数的线程池
- `newCachedThreadPool`: 创建线程数目动态增长的线程池。
- `newSingleThreadExecutor`: 创建只包含单个线程的线程池。
- `newScheduledThreadPool`: 设定 延迟时间后执行命令, 或者定期执行命令. 是进阶版的 `Timer`。

Executors 本质上是 `ThreadPoolExecutor` 类的封装。

`ThreadPoolExecutor` 提供了更多的可选参数, 可以进一步细化线程池行为的设定。

```
ThreadPoolExecutor(int corePoolSize, int maximumPoolSize, long keepAliveTime, TimeUnit unit,
BlockingQueue<Runnable> workQueue, ThreadFactory threadFactory, RejectedExecutionHandler handler)
Creates a new ThreadPoolExecutor with the given initial parameters.
```

- `corePoolSize`: 正式员工的数量. (正式员工, 一旦录用, 永不辞退)
- `maximumPoolSize`: 正式员工 + 临时工的数目. (临时工: 一段时间不干活, 就被辞退).
- `keepAliveTime`: 临时工允许的空闲时间。
- `unit`: `keepaliveTime` 的时间单位, 是秒, 分钟, 还是其他值。
- `workQueue`: 传递任务的阻塞队列
- `threadFactory`: 创建线程的工厂, 参与具体的创建线程工作. 通过不同线程工厂创建出的线程相当于对一些属性进行了不同的初始化设置。
- `RejectedExecutionHandler`: 拒绝策略, 如果任务量超出公司的负荷了接下来怎么处理。
 - `AbortPolicy()`: 超过负荷, 直接抛出异常。

- `CallerRunsPolicy()`: 调用者负责处理多出来的任务.
- `DiscardOldestPolicy()`: 丢弃队列中最老的任务.
- `DiscardPolicy()`: 丢弃新来的任务.

实现线程池

- 核心操作为 `submit`, 将任务加入线程池中
- 使用 `Worker` 类描述一个工作线程. 使用 `Runnable` 描述一个任务.
- 使用一个 `BlockingQueue` 组织所有的任务
- 每个 `worker` 线程要做的事情: 不停的从 `BlockingQueue` 中取任务并执行.
- 指定一下线程池中的最大线程数 `maxWorkerCount`; 当当前线程数超过这个最大值时, 就不再新增线程了.

```
1 class MyThreadPool {
2     private BlockingQueue<Runnable> queue = new LinkedBlockingQueue<>();
3
4     // 通过这个方法, 来把任务添加到线程池中.
5     public void submit(Runnable runnable) throws InterruptedException {
6         queue.put(runnable);
7     }
8
9     // n 表示线程池里有几个线程.
10    // 创建了一个固定数量的线程池.
11    public MyThreadPool(int n) {
12        for (int i = 0; i < n; i++) {
13            Thread t = new Thread(() -> {
14                while (true) {
15                    try {
16                        // 取出任务, 并执行~~
17                        Runnable runnable = queue.take();
18                        runnable.run();
19                    } catch (InterruptedException e) {
20                        e.printStackTrace();
21                    }
22                }
23            });
24            t.start();
25        }
26    }
27 }
```

```
1 // 线程池
2 public class Demo {
3     public static void main(String[] args) throws InterruptedException {
4         MyThreadPool pool = new MyThreadPool(4);
5         for (int i = 0; i < 1000; i++) {
6             pool.submit(new Runnable() {
7                 @Override
8                 public void run() {
9                     // 要执行的工作
10                    System.out.println(Thread.currentThread().getName() + " hell
11                }
12            });
13        }
14    }
15 }
```

9. 总结-保证线程安全的思路

1. 使用没有共享资源的模型
2. 适用共享资源只读，不写的模型
 - a. 不需要写共享资源的模型
 - b. 使用不可变对象
3. 直面线程安全（重点）
 - a. 保证原子性
 - b. 保证顺序性
 - c. 保证可见性

10. 对比线程和进程

10.1 线程的优点

1. 创建一个新线程的代价要比创建一个新进程小得多
2. 与进程之间的切换相比，线程之间的切换需要操作系统做的工作要少很多
3. 线程占用的资源要比进程少很多
4. 能充分利用多处理器的可并行数量

5. 在等待慢速I/O操作结束的同时，程序可执行其他的计算任务
6. 计算密集型应用，为了能在多处理器系统上运行，将计算分解到多个线程中实现
7. I/O密集型应用，为了提高性能，将I/O操作重叠。线程可以同时等待不同的I/O操作。

10.2 进程与线程的区别

1. 进程是系统进行资源分配和调度的一个独立单位，线程是程序执行的最小单位。
2. 进程有自己的内存地址空间，线程只独享指令流执行的必要资源，如寄存器和栈。
3. 由于同一进程的各线程间共享内存和文件资源，可以不通过内核进行直接通信。
4. 线程的创建、切换及终止效率更高。